RAG 어드민 대시보드

지표 모니터링 — 설계 · 구현 통합 PRD

v3.0 · 2025-03 | Ops · Customer · QA

✔ v2.0 대비 변경 사항

- 섹션 간 중복 제거 — 지표 정의는 마스터 테이블 1개로 통합
- 대시보드 "확장 지표" 탭 제거 → 각 섹션에 인라인 통합
- 3장 신설: 지표별 구체적 수집·계산 구현 방안 (코드 패턴, 도구 스택)
- 구현 로드맵에 의존성 및 P0 항목 명확화

1. 변경 이력

v1.0	2025-03	최초 작성 — 3섹션, 기본 지표 (Retrieval/Generation/성능)
v2.0	2025-03	7개 신규 영역 추가: 비용, 데이터 건강, 드리프트, 인용, 멀티턴, 안전성, 인프라
v3.0	2025-03	중복 제거 / 지표 마스터 단일화 / 지표별 구현 방안 챕터 신설

2. 지표 마스터 테이블 (전체)

모든 지표의 단일 출처(Single Source of Truth). 구현 상세는 3장을 참조한다.

2-1. Retrieval 품질

Recall@K (K=5,10)	> 0.85	1h	Ops+QA	RAGAS context_recall + Synthetic QA ground_truth
Precision@K	> 0.70	1h	QA	RAGAS context_precision
MRR / NDCG@K	> 0.75 / 0.80	평가 배치	QA	sklearn ndcg_score + 자체 MRR 스크립트
Similarity 분포 (P50)	P50 > 0.72	10min	Ops	Vector DB 반환 score 로그 히스토그램
검색 실패율 (sim<0.60)	< 10%	10min	Ops	score < 0.60 요청 카운터 / 전체 요청

2-2. Generation 품질

Faithfulness	> 0.90	1h	Ops+QA	RAGAS faithfulness (LLM Judge 내장)

Answer Relevance	> 0.85	1h	Ops+QA	RAGAS answer_relevancy
Answer Correctness	> 0.80	평가 배치	QA	RAGAS answer_correctness (ground_truth 필요)
Hallucination Rate	< 5%	1h	Ops	커스텀 LLM Judge — 환각 탐지 프롬프트 (3-2장)
Unanswerable Detection Rate	> 85%	1h	Ops+QA	LLM Judge — 거부 판단 정확도 분류

2-3. 시스템 성능

E2E Latency P95	< 3,000ms	1min	Ops	OTel span: 요청 인입 ~ 응답 완료
Retrieval / LLM Latency	< 500ms / 2,000ms	1min	Ops	OTel 단계별 span + Prometheus 히스토그램
TTFT (First Token)	< 800ms	1min	Ops	스트리밍 첫 청크 수신 타임스탬프
Error Rate	< 1%	1min	Ops	HTTP 5xx + 앱 예외 Counter (Prometheus)
QPS	추적	1min	Ops	Prometheus Counter / time.Duration

2-4. 비용 추적

Cost per Query	$0.002–0.008	1h	Ops	usage.tokens × 모델 단가 합산 → Prometheus Gauge
일 총 비용	예산 이내	1일	Ops	AWS/GCP Cost API 배치 수집 (LLM+Emb+Infra 태그 분리)
Cache Hit Rate	> 20%	1h	Ops	Redis SemanticCache HIT/MISS 카운터
Token Usage (in/out)	추적	1h	Ops	response.usage.prompt_tokens + completion_tokens

2-5. 데이터 건강

Index Freshness	< 24h 경과	실시간	Ops	수집 완료 이벤트 → Redis 타임스탬프 기록
Stale 문서 비율	< 5%	1일	Ops	Vector DB 메타: now - doc.updated_at > 90일 비율
Knowledge Gap Rate	< 8%	1h	Ops+Customer	검색 실패(sim<0.50) 쿼리 로그 카운터
Ingestion Error Rate	< 1%	실시간	Ops	파싱·임베딩 예외 이벤트 카운터
문서 중복 비율	< 3%	인덱싱 시	QA	Vector DB 배치: cosine > 0.98 청크 쌍 스캔

Embedding Drift (주간)	Δcosine < 0.10	1주	QA	임베딩 분포 KL divergence + UMAP 시각화

2-6. 드리프트 감지

Query Distribution Drift	cosine < 0.10	1h	Ops	7일 rolling 쿼리 임베딩 centroid 이탈 (cosine distance)
Retrieval Baseline Deviation	±5%p 이내	1h	Ops	rolling Recall@K 대비 현재 이탈량 퍼센트포인트
Query Novelty Rate	추적	1일	QA	신규 임베딩 클러스터 비율 (sklearn KMeans + 신규 클러스터 판정)
Model Behavior Drift	변동성 < 0.15	1주	QA	고정 프롬프트 10개 주기 실행 → BERTScore 분산

2-7. 인용 추적

Context Utilization Rate	> 60%	평가 배치	QA	검색 청크 vs LLM 사용 청크 비교 (LLM Judge JSON 반환)
Citation Coverage	> 70%	평가 배치	QA	출처 태그([1] 등) 포함 응답 비율 (정규식 + LLM Judge)
Citation Correctness	> 85%	평가 배치	QA	인용 출처 내용 ↔ 답변 내용 일치 (LLM Judge)
Source Diversity	평균 2–4개	평가 배치	QA	답변당 고유 source_id 수 카운트

2-8. 멀티턴 품질

Session Success Rate	> 75%	1일	Customer+QA	세션 종료 후 LLM Judge 성공 분류 (목표 달성 여부)
Context Retention Score	> 80%	평가 배치	QA	이전 턴 정보 반영 정확도 (Synthetic 멀티턴 + LLM Judge)
Reference Resolution Accuracy	> 75%	평가 배치	QA	지시어 해석 정확도 (멀티턴 QA 데이터셋 + LLM Judge)
Topic Coherence Score	> 0.75	평가 배치	QA	세션 내 쿼리 임베딩 silhouette score
Re-retrieval Trigger Rate	추적	평가 배치	QA	신규 검색 판단 정확도 (LLM Judge)

2-9. 안전성 / 컴플라이언스

PII Leakage Rate	0건 (즉시 알림)	실시간	Ops+QA	Presidio Analyzer 응답 스캔 → 감지 시 즉시 마스킹 + 알림
Out-of-Domain Rejection Rate	> 80%	1h	Ops+QA	OOD 테스트셋 + LLM Judge 거절 판정
Adversarial Resistance Rate	> 95%	배포 전	QA	레드팀 케이스셋 자동 실행 (pytest CI/CD 훅)
Toxicity / Safety Score	< 0.1%	1h	Ops	Detoxify 또는 Perspective API 분류

2-10. 인프라 효율

Vector DB QPS / Memory	Memory < 80%	1min	Ops	Vector DB Prometheus exporter 스크레이프
Embedding API Rate Limit Hit	< 0.1%	1min	Ops	HTTP 429 응답 카운터 / 전체 임베딩 요청
Fallback Rate (벡터→키워드)	< 5%	5min	Ops	폴백 경로 진입 카운터 / 전체 요청
Answer Rate	> 90%	5min	Ops	답변 생성 완료 카운터 / 전체 요청

3. 지표별 구현 방안

각 수집 방법의 구체적 구현 절차, 코드 패턴, 도구 설정을 기술한다.

3-1. RAGAS 프레임워크 (Faithfulness · Relevance · Recall · Precision · Correctness)

RAGAS는 RAG 전용 오픈소스 평가 라이브러리다. LLM을 judge로 사용하여 참조 답안 없이도 대부분의 지표를 산출할 수 있다.

설치

```
pip install ragas langchain anthropic
```

평가 실행 패턴

```
from ragas import evaluate
from ragas.metrics import (faithfulness, answer_relevancy,
                           context_recall, context_precision,
                           answer_correctness)
from datasets import Dataset

data = {
  "question":     [q1, q2, ...],        # 사용자 질문
  "answer":       [a1, a2, ...],        # LLM 생성 답변
  "contexts":     [[c1a, c1b], ...],    # 검색된 청크 리스트
  "ground_truth": [gt1, gt2, ...],      # Synthetic QA 정답 (선택)
}
result = evaluate(Dataset.from_dict(data),
                  metrics=[faithfulness, answer_relevancy,
                           context_recall, context_precision])
# result["faithfulness"] → 0.0-1.0 float
```

배치 실행 전략

- 프로덕션 트래픽 5–10% 샘플링 → 1시간 배치로 평가 실행
- 평가 결과 → TimescaleDB 또는 InfluxDB 적재 → Grafana 연동
- Anthropic Batch API 활용으로 동기 호출 대비 약 50% 비용 절감

3-2. LLM-as-Judge (Hallucination · Unanswerable · Citation · Session Success)

Claude 또는 GPT-4o를 평가자로 사용하여 구조화된 JSON으로 판정 결과를 수집한다. temperature=0, few-shot 예시 3–5개를 시스템 프롬프트에 포함한다.

환각 탐지 프롬프트 패턴

```
SYSTEM = """
당신은 RAG 응답 품질 평가자입니다. 반드시 JSON만 출력하세요.
[컨텍스트]에 없는 사실이 [답변]에 있으면 hallucinated=true 입니다.
출력 형식: {"hallucinated": bool, "reason": "...", "score": 0.0-1.0}
"""

USER = f"""[컨텍스트]: {retrieved_chunks}
[답변]: {llm_answer}"""
```

배치 비용 최적화

- Anthropic Batch API: /v1/messages/batches 엔드포인트, 동기 50% 할인
- 캐싱: (question_hash + answer_hash) → 동일 평가 재사용
- 소형 모델(claude-haiku) 1차 필터 → 경계 케이스만 대형 모델 재판정

3-3. APM 계측 — OpenTelemetry + Prometheus

파이프라인 각 단계에 OTel span을 삽입하고 Prometheus로 집계한다. Datadog, Grafana Cloud, 자체 Prometheus + Grafana 모두 호환된다.

파이프라인 계측 패턴 (Python)

```
from opentelemetry import trace
from prometheus_client import Counter, Histogram

tracer = trace.get_tracer(__name__)

RAG_LATENCY = Histogram("rag_latency_seconds", "단계별 레이턴시",
  ["stage"], buckets=[.1, .25, .5, 1, 2, 3, 5, 10])
RAG_ERRORS  = Counter("rag_errors_total", "에러 수", ["stage"])
RAG_TOKENS  = Counter("rag_tokens_total", "토큰 수", ["type", "customer_id"])

def rag_query(question: str, customer_id: str):
  with tracer.start_as_current_span("rag.retrieval"):
    with RAG_LATENCY.labels("retrieval").time():
      chunks = retriever.get(question)
  with tracer.start_as_current_span("rag.llm"):
    with RAG_LATENCY.labels("llm").time():
      resp = llm.generate(question, chunks)
      RAG_TOKENS.labels("input",  customer_id).inc(resp.usage.input_tokens)
      RAG_TOKENS.labels("output", customer_id).inc(resp.usage.output_tokens)
  return resp
```

3-4. 비용 수집 및 Semantic Cache

쿼리당 비용 계산

```
PRICING = {  # USD per token (2025-03 기준)
  "claude-sonnet-4-6": {"in": 3e-6,  "out": 15e-6},
  "claude-haiku-4-5":  {"in": 0.8e-6, "out": 4e-6},
}

def record_cost(resp, model: str, customer_id: str) -> float:
  cost = (resp.usage.input_tokens  * PRICING[model]["in"] +
          resp.usage.output_tokens * PRICING[model]["out"])
  COST_GAUGE.labels(customer_id=customer_id).inc(cost)
  return cost
```

Semantic Cache (Redis)

```
from langchain_community.cache import RedisSemanticCache

cache = RedisSemanticCache(
  redis_url="redis://localhost:6379",
  embedding=embedder,
  score_threshold=0.97,   # cosine > 0.97 → HIT
)
# Cache Hit Rate = HIT_count / (HIT + MISS) 카운터로 추적
```

3-5. 데이터 건강 수집

Index Freshness

```
# 수집 완료 시 이벤트 훅에서 호출
def on_ingestion_done(customer_id: str):
  redis.set(f"freshness:{customer_id}", time.time())

# 대시보드 쿼리
def get_freshness_hours(customer_id: str) -> float:
  ts = redis.get(f"freshness:{customer_id}")
  return (time.time() - float(ts)) / 3600 if ts else float("inf")
```

Knowledge Gap 탐지

```
def on_query_complete(question, chunks, max_sim):
  if not chunks or max_sim < 0.50:
    KNOWLEDGE_GAP_COUNTER.labels(customer_id).inc()
    gap_db.insert({"question": question, "ts": time.time()})
```

3-6. 드리프트 감지 구현

Query Distribution Drift — Rolling Centroid

```
from collections import deque
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

class QueryDriftDetector:
  def __init__(self, window_size=7*24*60):  # 7일, 1분 버킷
    self.buf = deque(maxlen=window_size)

  def update(self, emb: np.ndarray):
    self.buf.append(emb)
    if len(self.buf) >= 120:          # 최소 2시간
      baseline = np.mean(list(self.buf)[:-60], axis=0)
      recent   = np.mean(list(self.buf)[-60:],  axis=0)
```

```
    drift = 1 - cosine_similarity([baseline],[recent])[0][0]
    DRIFT_GAUGE.set(float(drift))
    if drift > 0.10: alert.warning("QUERY_DRIFT", value=drift)
```

3-7. 안전성 수집 구현

PII 감지 — Microsoft Presidio

```
pip install presidio-analyzer presidio-anonymizer

from presidio_analyzer  import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

analyzer  = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def scan_and_mask(answer: str, customer_id: str) -> str:
  hits = analyzer.analyze(text=answer, language="ko")
  # 한국어 커스텀 recognizer 추가 필요 → Open Issue #6
  if hits:
    PII_COUNTER.labels(customer_id).inc(len(hits))
    alert.critical("PII_DETECTED", customer_id=customer_id)
    return anonymizer.anonymize(answer, hits).text
  return answer
```

레드팀 자동화 — CI/CD 연동

- 케이스셋 구성: 프롬프트 인젝션(60건), 역할극 탈출(30건), 민감 유도(50건)
- pytest fixture로 케이스셋 로드 → LLM Judge 판정 → 통과율 assert
- GitHub Actions: PR 머지 전 실행, 통과율 < 95% 시 배포 블록
- 신규 실패 케이스 자동 추가 → 회귀 방지

3-8. Synthetic QA 생성 (Ground Truth 확보)

RAGAS TestsetGenerator로 지식베이스에서 Q&A 쌍을 자동 생성하여 ground_truth를 확보한다. context_recall, answer_correctness 산출에 필수다.

단일 QA 생성

```
from ragas.testset.generator   import TestsetGenerator
from ragas.testset.evolutions  import simple, reasoning, multi_context

gen = TestsetGenerator.from_langchain(
  generator_llm=claude_llm, critic_llm=claude_llm, embeddings=embedder)
testset = gen.generate_with_langchain_docs(
  docs, test_size=200,
  distributions={simple: 0.4, reasoning: 0.4, multi_context: 0.2})
```

멀티턴 QA 확장

- 1턴 답변을 컨텍스트로 추가한 2·3턴 질문 자동 생성 (LLM 확장)
- 지시어 삽입: "그것", "위의 내용", "방금 설명한" 등 의도적 포함
- 생성 주기: 지식베이스 대규모 업데이트 시 + 월 1회 전체 재생성
- 카테고리 × 난이도(쉬움/중간/어려움) 레이블 자동 부여 후 QA 탭 히트맵에 표시

4. 대시보드 섹션 스펙
패널 목록과 데이터 소스를 정의한다. 지표 정의는 2장, 구현 방법은 3장 참조.

4-1. Ops Monitor (내부 엔지니어)

상태 헤더 KPI	Latency P95 / Hallucination / Error Rate / Cost/Query / Answer Rate	1min	Prometheus + RAGAS 배치
실시간 트래픽	P50/P90/P95/P99 레이턴시, QPS, Error Rate 추이	1min	Prometheus
파이프라인 병목	단계별 레이턴시 (Retrieval/Reranking/LLM/후처리)	5min	OpenTelemetry spans
Retrieval 품질	Similarity 분포 히스토그램, Recall@K 추이, 검색 실패율	10min	Vector DB + RAGAS
Generation 품질	Faithfulness 게이지, Hallucination 추이, Answer Relevance	1h	RAGAS + LLM Judge
비용 추적	Cost/Query 스파크라인, 일별 비용 스택 (LLM+Emb+Infra), Cache Hit Rate	1h	LLM usage + Redis
데이터 건강	Index Freshness, Stale 비율, Knowledge Gap, Ingestion Error Rate	실시간	파이프라인 이벤트 + Redis
드리프트 감지	Query Drift (cosine), Baseline Deviation (%p), Novelty Rate	1h	자체 드리프트 스크립트
안전성 실시간	PII Leakage, Unanswerable Rate, OOD Rejection, Safety Score	실시간	Presidio + LLM Judge
인프라 효율	Vector DB Memory/QPS, Fallback Rate, Answer Rate, Rate Limit Hit	1min	Prometheus exporter
알림 로그	활성 알림 배너 + 이력 테이블 (심각도·지표·값·해소 시각), CSV 다운로드	실시간	AlertManager

- 권한: 내부 엔지니어 전용 — 비용 상세, 고객사 원시 로그, OTel 트레이스 링크 포함
- 알림 라우팅: Critical → PagerDuty 온콜, Warning → Slack #rag-alerts

4-2. Customer Report (고객사 · CS팀)

KPI 카드 5개	평균 응답속도 / 답변 정확도 / 월간 질의 수 / 만족도 (👍%) / SLA 준수율	1일	일 배치 집계 DB
월별 품질 트렌드	Faithfulness + Relevance 6개월 영역 라인차트	1일	RAGAS 배치
시간대별 사용량	시간대별 질의 분포 막대차트 (피크 강조)	1일	로그 집계
질의 유형 분포	카테고리 도넛 + 건수 테이블	1일	LLM 분류 배치
만족도 추이	주별 👍/👎 비율 12주 라인차트	1일	피드백 DB
SLA 리포트	P95 준수율 게이지 + 위반 이력 테이블, PDF 다운로드	1일	Prometheus

세션 성공률	월별 성공률 + 평균 세션 길이 + 미해결 Top 원인	1일	LLM Judge 배치
미커버 질문 현황	미답변 Top 20, 카테고리 분포, 처리 상태 (담당자 업데이트 가능)	주 1회	Knowledge Gap DB
서비스 안전성 요약	안전성 종합 점수 (100점 만점), 이달 이벤트 건수	월 1회	안전성 배치 집계

- 권한: 고객사별 customer_id 필터 강제 적용, PII 마스킹, 비용 상세·원시 로그 비공개

4-3. QA Evaluation (QA팀 · 엔지니어)

레이더 스코어카드	5축 (Faithfulness/Relevance/Correctness/Recall@K/Precision@K), 버전 오버레이	평가 배치	RAGAS
버전 비교	그룹 막대차트 + 델타 테이블 (p-value)	평가 배치	RAGAS
Retrieval 심층 분석	쿼리 복잡도 vs Recall 산점도, Chunk Similarity 히스토그램	평가 배치	RAGAS + Vector DB
Hallucination 분석	유형 도넛 (컨텍스트 부족/모델 오류/청크 오염/기타) + 샘플 테이블	평가 배치	LLM Judge
Synthetic QA 히트맵	카테고리 × 난이도(쉬움/중간/어려움) 정답률 매트릭스	평가 배치	RAGAS + Synthetic QA
인용 추적	CTX Utilization / Citation Coverage / Correctness 막대 + Source Diversity 히스토그램	평가 배치	LLM Judge
멀티턴 품질	Context Retention / Ref. Resolution / Topic Coherence / Re-retrieval Rate + 경고 배너	평가 배치	LLM Judge + 멀티턴 QA
안전성 레드팀	5개 항목 통과율 테이블 + 배포 이력, CI/CD 결과 자동 기록	배포 전	레드팀 스크립트
데이터 건강 심층	청크 품질 분포, Embedding Drift UMAP 산점도, Knowledge Gap 클러스터	주 1회	Vector DB + UMAP
A/B 실험	워터폴 차트 + 통계 유의성 테이블 (채택/기각)	실험 종료 시	실험 배치
LLM Judge 분포	박스플롯 점수 분포 + 이상치 케이스 드릴다운 모달	평가 배치	LLM Judge

- 권한: 내부 QA팀 + 엔지니어. 실험 편집·배포 블록 해제 권한은 별도 RBAC 롤

5. 알림 임계값 통합 기준표

AlertManager 룰 파일 또는 대시보드 설정에서 이 값을 직접 참조한다.

E2E Latency P95	> 2,500ms	> 3,000ms	LLM/Retrieval 단계 확인 → 스케일아웃 (자동화 가능)
Error Rate	> 0.5%	> 1%	오류 로그 확인, 서킷브레이커 트리거 (자동화 가능)
Hallucination Rate	> 3%	> 5%	프롬프트 롤백 또는 컨텍스트 품질 점검 (수동)
Recall@K	< 0.80	< 0.70	임베딩 모델·청크 사이즈·인덱스 재검토 (수동)
Faithfulness	< 0.85	< 0.80	LLM Judge 샘플 확인, 프롬프트 수정 (수동)
Cost per Query	> $0.010	> $0.020	토큰 패턴 분석, 프롬프트 압축 검토 (수동)
Cache Hit Rate	< 15%	< 5%	캐시 키 전략 재검토 (수동)
Index Freshness	> 24h 경과	> 72h 경과	수집 파이프라인 상태 점검 (자동화 가능)
Knowledge Gap Rate	> 8%	> 15%	미커버 주제 리뷰 및 지식베이스 확충 (수동)
Ingestion Error Rate	> 1%	> 3%	파싱 오류 로그 확인 (자동화 가능)
Query Distribution Drift	cosine > 0.10	cosine > 0.20	쿼리 패턴 분석 → 임베딩 재훈련 검토 (수동)
Retrieval Baseline Dev	> ±5%p	> ±10%p	즉시 회귀 테스트 실행 (자동화 가능)
PII Leakage Rate	> 0건 즉시	N/A	세션 격리 + 원인 조사 (자동화 필수)
Unanswerable Detection	< 80%	< 70%	거부 판단 프롬프트 재검토 (수동)
Fallback Rate	> 8%	> 15%	Vector DB 인덱스 상태 점검 (수동)
Answer Rate	< 88%	< 80%	No-result 원인 분석 (수동)

6. 구현 로드맵

P0	OpenTelemetry 계측 전체 파이프라인 삽입	3일	없음
P0	Prometheus 메트릭 정의 + Grafana/Datadog 연동	2일	OTel 완료
P0	비용 수집 (usage 태깅, customer_id 분리)	1일	OTel 완료
P0	Ops 상태 헤더 + 트래픽 + 파이프라인 병목 패널	3일	Prometheus 완료
P0	Index Freshness + Ingestion Error Rate 수집	2일	수집 파이프라인 이벤트 훅
P1	Synthetic QA 최초 생성 (200건) + RAGAS 배치 파이프라인	4일	Claude API 키
P1	LLM-as-Judge 배치 (Hallucination · Unanswerable)	3일	프롬프트 확정 (Open Issue #1)
P1	Query Drift 감지 스크립트 + Ops 드리프트 패널	3일	임베딩 로그 수집

P1	Presidio PII 스캐너 연동 + 안전성 실시간 패널	3일	한국어 recognizer 결정 (Open Issue #6)
P1	Customer 기본 패널 (KPI·트렌드·SLA)	4일	일 배치 집계 스키마
P2	Redis SemanticCache + Cache Hit Rate 추적	2일	Redis 인프라
P2	인프라 효율 패널 (Vector DB exporter, Fallback, Answer Rate)	2일	Prometheus exporter
P2	QA 레이더·버전 비교 ·Hallucination 분석 패널	4일	RAGAS 배치 완료
P2	Customer 세션 성공률 + Knowledge Gap 패널	3일	LLM Judge 세션 분류
P3	인용 추적 파이프라인 + QA 인용 패널	4일	Citation 평가 프롬프트
P3	멀티턴 Synthetic QA 생성 + 멀티턴 품질 패널	5일	멀티턴 QA 데이터셋
P3	레드팀 자동화 케이스셋 + CI/CD pytest 훅	4일	레드팀 케이스셋 (Open Issue #2)
P3	Embedding Drift UMAP 시각화 (주간 배치)	3일	UMAP + 컴퓨팅 예산 (Open Issue #9)

7. 미결 사항 (Open Issues)

1	LLM Judge 평가 모델: Claude claude-sonnet-4-6 vs GPT-4o vs 자체 파인튜닝. 비용·일관성 트레이드오프	엔지니어링 리드	P1 LLM Judge 배치
2	레드팀 케이스셋 초기 구성 책임자 및 최소 케이스 수 기준	QA팀	P3 레드팀 CI/CD
3	Customer Report 고객사 접근 방식: 별도 로그인 vs 어드민 서브뷰 RBAC	PM + 보안	P1 Customer 패널
4	Slack/PagerDuty 알림 채널 구조 및 온콜 로테이션 연동 범위	운영팀	P0 알림 로그
5	지표 데이터 보존: 원시 로그(30일) / 집계(1년) / 평가(영구) 정책	인프라팀	전체 스토리지
6	한국어 PII Presidio 커스텀 recognizer 직접 구현 여부 (주민번호·계좌 등)	보안팀	P1 PII 스캐너
7	비용 태깅 전략: 고객사별 분리 집계 및 외부 청구 연동 방식	인프라팀 + PM	P0 비용 수집
8	멀티턴 Synthetic QA 생성 주기 및 최초 데이터셋 구축 주체	QA팀	P3 멀티턴 패널
9	UMAP 계산 주기 및 서버 비용 정책 (GPU 필요 여부)	인프라팀	P3 Drift UMAP